

Ansible - Flow control

Mastering flow control, conditionals, and error handling for robust automation

Overview

Building Sophisticated Playbooks



This guide explores advanced Ansible techniques that transform basic playbooks into powerful, resilient automation tools. You'll learn to control execution flow, handle errors gracefully, and write more maintainable code.

These features enable you to build production-ready playbooks that adapt to different environments, recover from failures, and execute with precision.

Flow Control with Loops

Loops allow you to execute the same task multiple times with different values, making playbooks more compact and flexible. The current iteration value is stored in the `item` variable by default.

You can iterate over lists, dictionaries (using `dict2items`), or hashes. For nested loops, use `loop_control.loop_var` to rename the iteration variable and avoid conflicts.



Loop over lists

Iterate through simple arrays of values



Loop over dictionaries

Process key-value pairs with `dict2items`



Nested loops

Use custom `loop_var` for complex iterations

Capturing Task Output

01

Register the variable

Use the `register` keyword to capture task output into a variable

02

Access the details

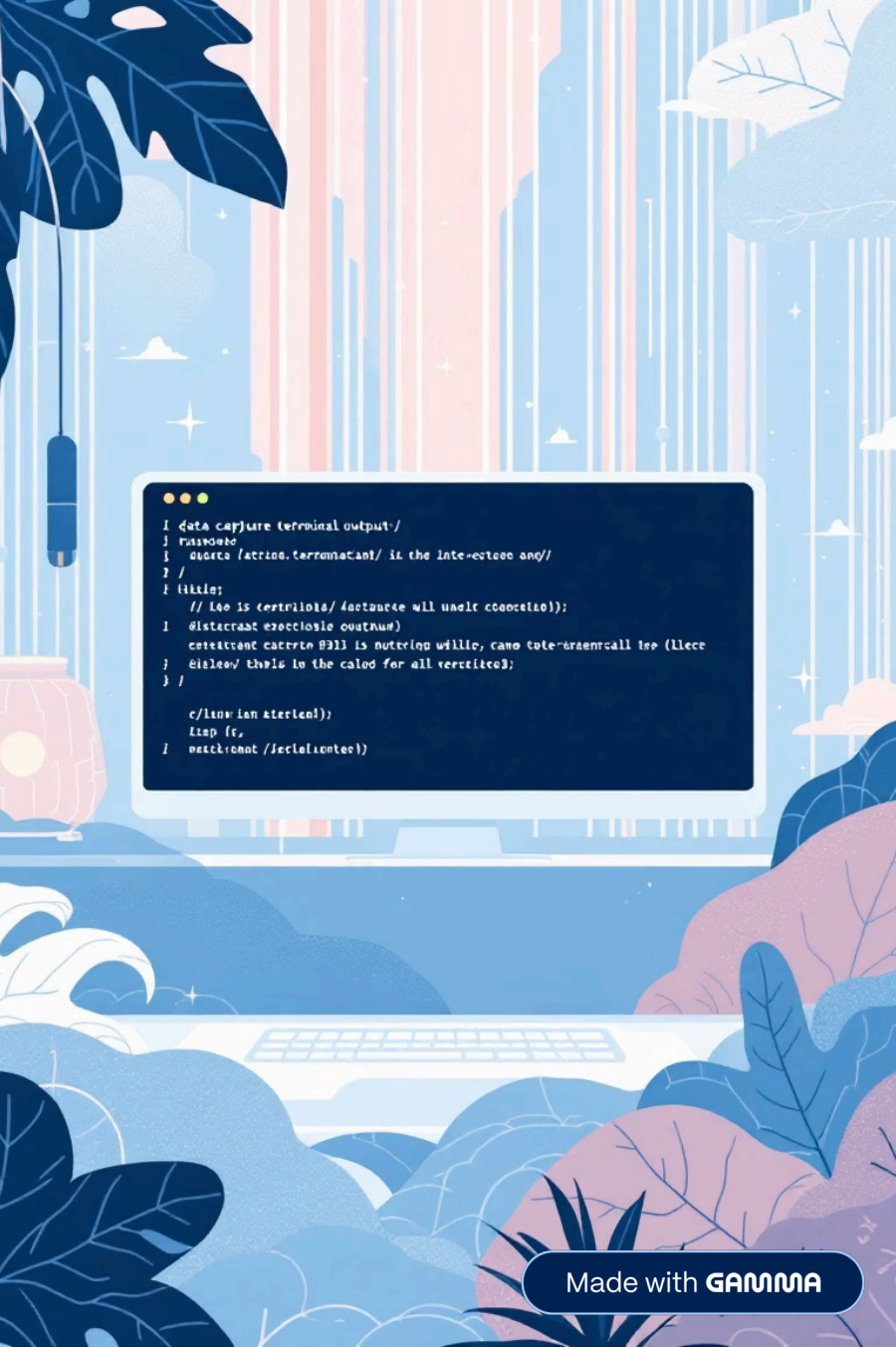
The variable contains `stdout`, `stderr`, `rc` (return code), and more

03

Use in subsequent tasks

Reference captured data with dot notation for conditional execution or reporting

- ❏ **Pro tip:** Registered variables persist throughout the play and can be used in multiple subsequent tasks, making them invaluable for complex workflows.



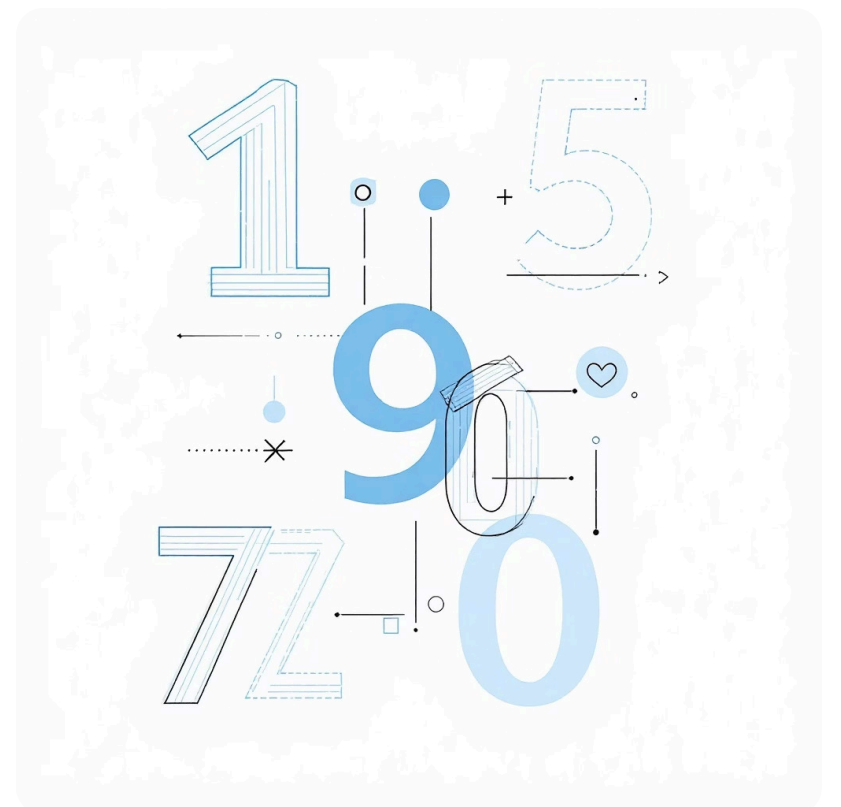
```
1 data capture (terminal output) {
2   register
3   quote (string, terminate) if the int-escape and/
4   /
5   title;
6   // Leo is (extrilola) (action) will under (conceal);
7   @istacrat exoclosio outnum)
8   extatant catrin 9333 is nutring willie, cano tote-ardemcall too (llec
9   @ialen/ this is the calod for all verctice);
10 }
11
12 c/linn ian aterlen);
13 stop {r,
14 }
15 }
16 }
17 }
18 }
19 }
20 }
21 }
22 }
23 }
24 }
25 }
26 }
27 }
28 }
29 }
30 }
```

Advanced Loop Control

Pausing Execution

The `pause` module halts playbook execution for a specified duration or awaits user input. Use the `seconds` parameter for timed pauses or `prompt` to display messages during the pause.

This proves essential when waiting for external processes to complete or requiring user confirmation before proceeding with critical operations.



```
- name: Pause for 5 seconds
  pause:
    seconds: 5

- name: Pause for user confirmation
  pause:
    prompt: "Continue with critical operation? (y/n)"
    echo: yes
```

Indexing Loop Iterations

Store the current loop index using `loop_control.index_var` in a custom variable. This variable can be referenced within tasks to display or utilise the iteration number for logging, naming, or conditional logic.

```
- name: Process items with index
  debug:
    msg: "Processing item {{ item }} (index: {{ my_idx }})"
  loop:
    - apple
    - banana
    - cherry
  loop_control:
    index_var: my_idx
```

Working with Special Variables

Ansible provides powerful built-in variables for advanced host and loop management.



Host list variables

`ansible_play_hosts`,
`ansible_play_batch`, and `groups`
contain host lists that can be used in
loops.

Example: `loop: "{{ groups['webservers'] }}"`



Dynamic host queries

The `query('inventory_hostnames',
pattern)` function retrieves hosts
dynamically from inventory using
patterns.



Loop metadata

With `loop_control: extended: true`,
access variables like
`ansible_loop.index`,
`ansible_loop.first`, and
`ansible_loop.last` for advanced
control.

Retrying Tasks Until Success



Ansible can automatically retry tasks until a specific condition is satisfied, essential for handling intermittent failures or waiting for services to become available.

Use `until`, `retries`, and `delay` to define the success condition, maximum number of attempts, and waiting time between tries.



 **Best practice:** Use retry logic with idempotent modules, as repeated execution of non-idempotent modules may cause inconsistent states.



Conditionals and Jinja2

Ansible uses the `when` directive to control task execution based on facts, variables, or registered results. This proves particularly useful when managing multiple OS types with different package managers.

Comparison operators

`==, !=, <, >, <=, >=, in, is`

Logical operators

`and, or, not` for combining multiple conditions

Jinja2 expressions

Evaluated as Python expressions with full filter and test support

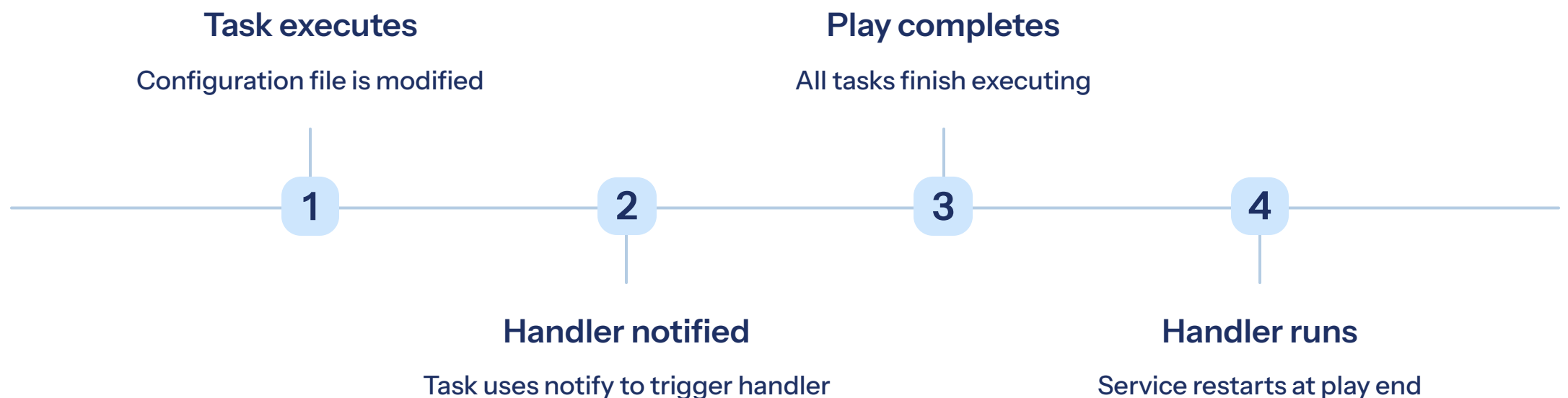
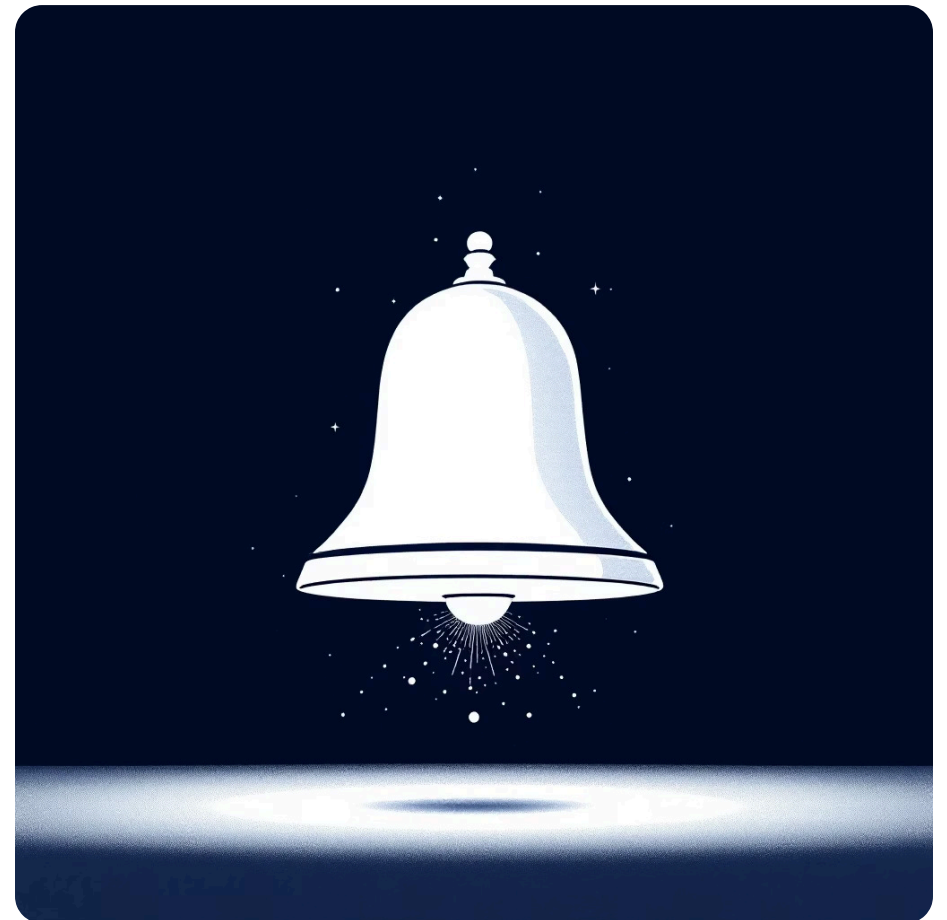
```
when: ansible_facts['os_family'] == "RedHat" and
      ansible_facts['distribution_major_version'] | int >= 8
```


Handlers and Event-Driven Actions

Handlers are special tasks that execute only when notified by other tasks. They typically perform actions like restarting services after configuration changes.

Handlers are defined in a separate section and triggered using the `notify` keyword. They run at the end of a play, ensuring all changes are applied before restarting services.

You can notify multiple handlers from a single task, and handlers can notify other handlers, creating sophisticated event chains.



Robust Error Management

Ignoring failures

Use `ignore_errors: yes` to continue playbook execution even if a task fails. Useful for non-critical tasks where failure shouldn't halt the entire playbook.

Defining failure conditions

Use `failed_when` to specify custom failure conditions based on task output or other criteria. Example:
`failed_when: "'Error' in myvar.stdout"`

Defining change status

Use `changed_when` to control when a task reports as changed. Enables more accurate reporting for tasks that don't always result in changes.

Aborting on failure

Set `any_errors_fatal: true` at play level to abort the entire play if any task fails. Essential for critical workflows.

Blocks with rescue and always

Group tasks in blocks with `rescue` sections for fallback actions and `always` sections for cleanup tasks that run regardless of success or failure.